Министерство Образования Республики Беларусь
УО Брестский Государственный Технический Университет
Кафедра ИИТ

Лабораторная работа № 3

По СПП

Вариант 4

Выполнила:
Студент 3 курса
Группы ПО-12
Езерская О.Г.
Проверил:
Кулик А.Д.

Брест 2026

Цель работы: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.

Задание 1

Первая группа заданий (порождающий паттерн)

Проект «Туристическое бюро».Реализовать возможность выбора программы тура (проезд, проживание, питание, посещение музеев, выставок, экскурсии и т.д.).Должна формироваться итоговая стоимость заказа.

```
Введите количество дней тура: 7

----------------------------------------
ВИДЫ ТРАНСПОРТА:
1.  Автобус (5000 руб)
2.  Поезд (8000 руб)
3.  Самолет (15000 руб)
4.  Без транспорта
Выберите транспорт (1-4): 1

----------------------------------------
ТИПЫ ПРОЖИВАНИЯ (ЦЕНА ЗА НОЧЬ):
1.  Хостел (2000 руб/ночь)
2.  Отель 3* (3500 руб/ночь)
3.  Отель 4* (5000 руб/ночь)
4.  Отель 5* (8000 руб/ночь)
5.  Без проживания
Выберите тип проживания (1-5): 3

----------------------------------------
ТИПЫ ПИТАНИЯ (ЦЕНА ЗА ДЕНЬ):
1.  Без питания (0 руб/день)
2.  Только завтрак (1000 руб/день)
3.  Полупансион (2000 руб/день)
4.  Полный пансион (3000 руб/день)
5.  Все включено (5000 руб/день)
Выберите тип питания (1-5): 4

----------------------------------------
ДОПОЛНИТЕЛЬНЫЕ УСЛУГИ:

1. Добавить музей (500 руб)
2.  Добавить выставку (400 руб)
3.  Добавить экскурсию (800 руб)
4. Завершить создание тура
Выберите услугу (1-4): 1
Введите название музея: Питер
 Музей 'Питер' добавлен!
```

```
1. Добавить музей (500 руб)
2.  Добавить выставку (400 руб)
3.  Добавить экскурсию (800 руб)
4. Завершить создание тура
Выберите услугу (1-4): 4

==================================================
 ВАШ ИНДИВИДУАЛЬНЫЙ ТУР
==================================================
 Транспорт: автобус
 Проживание: отель 4* (7 ночей)
 Питание: полный пансион (7 дней)
Музеи: Питер
 Выставки: не выбраны
 Экскурсии: не выбраны
 ИТОГОВАЯ СТОИМОСТЬ: 61500 руб.
==================================================

✨ Спасибо за использование нашего сервиса! ✨
```

```
from abc import ABC, abstractmethod

from datetime import datetime


class Tour:
```

```
def __init__(self):
    self.transport = None
    self.accommodation = None
    self.meals = None
    self.museums = []
    self.exhibitions = []
    self.excursions = []
    self.price = 0

def __str__(self):
    result = "\n" + "="*50
    result += "\n ВАШ ИНДИВИДУАЛЬНЫЙ ТУР "
    result += "\n" + "="*50
    result += f"\n Транспорт: {self.transport or 'не выбран'}"
    result += f"\n Проживание: {self.accommodation or 'не выбрано'}"
    result += f"\n Питание: {self.meals or 'не выбрано'}"

    if self.museums:
        result += f"\nМузеи: {', '.join(self.museums)}"
    else:
        result += "\n Музеи: не выбраны"

    if self.exhibitions:
        result += f"\n Выставки: {', '.join(self.exhibitions)}"
    else:
        result += "\n Выставки: не выбраны"

    if self.excursions:
        result += f"\n Экскурсии: {', '.join(self.excursions)}"
    else:
```

```
            result += "\n Экскурсии: не выбраны"

        result += f"\n ИТОГОВАЯ СТОИМОСТЬ: {self.price} руб."
        result += "\n" + "="*50
        return result

class TourBuilder(ABC):
    @abstractmethod
    def reset(self):
        pass

    @abstractmethod
    def build_transport(self, transport_type):
        pass
    @abstractmethod
    def build_accommodation(self, accommodation_type, nights):
        pass
    @abstractmethod
    def build_meals(self, meals_type, days):
        pass
    @abstractmethod
    def build_museum(self, museum_name):
        pass
    @abstractmethod
    def build_exhibition(self, exhibition_name):
        pass
    @abstractmethod
    def build_excursion(self, excursion_name):
        pass
```

```
    @abstractmethod
    def get_tour(self):
        pass
class TourBuilderImpl(TourBuilder):
    def __init__(self):
        self.tour = None
        self.reset()

    def reset(self):
        self.tour = Tour()

    def build_transport(self, transport_type):
        transport_prices = {
            "автобус": 5000,
            "поезд": 8000,
            "самолет": 15000,
            "нет": 0
        }
        self.tour.transport = transport_type
        self.tour.price += transport_prices.get(transport_type.lower(), 0)

    def build_accommodation(self, accommodation_type, nights):
        accommodation_prices = {
            "хостел": 2000,
            "отель 3*": 3500,
            "отель 4*": 5000,
            "отель 5*": 8000,
            "нет": 0
        }
        price_per_night = accommodation_prices.get(accommodation_type.lower(), 0)
```

```
        self.tour.accommodation = f"{accommodation_type} ({nights} ночей)"
        self.tour.price += price_per_night * nights

    def build_meals(self, meals_type, days):
        meals_prices = {
            "без питания": 0,
            "завтрак": 1000,
            "полупансион": 2000,
            "полный пансион": 3000,
            "все включено": 5000
        }
        price_per_day = meals_prices.get(meals_type.lower(), 0)
        self.tour.meals = f"{meals_type} ({days} дней)"
        self.tour.price += price_per_day * days

    def build_museum(self, museum_name):
        self.tour.museums.append(museum_name)
        self.tour.price += 500

    def build_exhibition(self, exhibition_name):
        self.tour.exhibitions.append(exhibition_name)
        self.tour.price += 400

    def build_excursion(self, excursion_name):
        self.tour.excursions.append(excursion_name)
        self.tour.price += 800

    def get_tour(self):
        tour = self.tour
        self.reset()
```

```
        return tour
def main():
    print("="*60)
    print("ТУРИСТИЧЕСКОЕ БЮРО 'ИНДИВИДУАЛЬНЫЙ ТУР' ")
    print("="*60)
    print("Создайте свой идеальный тур шаг за шагом!")

    builder = TourBuilderImpl()

    # Ввод количества дней
    while True:
        try:
            days = int(input("\nВведите количество дней тура: "))
            if days > 0:
                break
            else:
                print("Количество дней должно быть положительным числом!")
        except ValueError:
            print("Пожалуйста, введите целое число!")
    print("\n" + "-"*40)
    print("ВИДЫ ТРАНСПОРТА:")
    print("1.  Автобус (5000 руб)")
    print("2.  Поезд (8000 руб)")
    print("3.  Самолет (15000 руб)")
    print("4.  Без транспорта")

    while True:
        transport_choice = input("Выберите транспорт (1-4): ")
        transport_map = {
            "1": "автобус",
```

```
        "2": "поезд",
        "3": "самолет",
        "4": "нет"
    }
    if transport_choice in transport_map:
        builder.build_transport(transport_map[transport_choice])
        break
    else:
        print("Неверный выбор! Пожалуйста, выберите 1-4.")
   print("\n" + "-"*40)
print("ТИПЫ ПРОЖИВАНИЯ (ЦЕНА ЗА НОЧЬ):")
print("1.  Хостел (2000 руб/ночь)")
print("2.  Отель 3* (3500 руб/ночь)")
print("3.  Отель 4* (5000 руб/ночь)")
print("4.  Отель 5* (8000 руб/ночь)")
print("5.  Без проживания")

while True:
    accom_choice = input("Выберите тип проживания (1-5): ")
    accom_map = {
        "1": "хостел",
        "2": "отель 3*",
        "3": "отель 4*",
        "4": "отель 5*",
        "5": "нет"
    }
    if accom_choice in accom_map:
        builder.build_accommodation(accom_map[accom_choice], days)
        break
    else:
```

```
        print("Неверный выбор! Пожалуйста, выберите 1-5.")
print("\n" + "-"*40)
print("ТИПЫ ПИТАНИЯ (ЦЕНА ЗА ДЕНЬ):")
print("1.  Без питания (0 руб/день)")
print("2.  Только завтрак (1000 руб/день)")
print("3.  Полупансион (2000 руб/день)")
print("4.  Полный пансион (3000 руб/день)")
print("5.  Все включено (5000 руб/день)")

while True:
    meals_choice = input("Выберите тип питания (1-5): ")
    meals_map = {
        "1": "без питания",
        "2": "завтрак",
        "3": "полупансион",
        "4": "полный пансион",
        "5": "все включено"
    }
    if meals_choice in meals_map:
        builder.build_meals(meals_map[meals_choice], days)
        break
    else:
        print("Неверный выбор! Пожалуйста, выберите 1-5.")

print("\n" + "-"*40)
print("ДОПОЛНИТЕЛЬНЫЕ УСЛУГИ:")

while True:
    print("\n1. Добавить музей (500 руб)")
    print("2.  Добавить выставку (400 руб)")
```

```
        print("3.  Добавить экскурсию (800 руб)")
        print("4. Завершить создание тура")

        service = input("Выберите услугу (1-4): ")

        if service == "1":
            museum = input("Введите название музея: ")
            builder.build_museum(museum)
            print(f" Музей '{museum}' добавлен!")

        elif service == "2":
            exhibition = input("Введите название выставки: ")
            builder.build_exhibition(exhibition)
            print(f" Выставка '{exhibition}' добавлена!")

        elif service == "3":
            excursion = input("Введите название экскурсии: ")
            builder.build_excursion(excursion)
            print(f" Экскурсия '{excursion}' добавлена!")

        elif service == "4":
            break
        else:
            print("Неверный выбор! Пожалуйста, выберите 1-4.")
    tour = builder.get_tour()
    print(tour)
    print("\n✨ Спасибо за использование нашего сервиса! ✨")

if __name__ == "__main__":
    main()
```

Задание 2

Вторая группа заданий (структурный паттерн)

Проект «Файловая система». Реализуйте модель работы файловой системы. Должна поддерживаться иерархичность ФС на уровне директорий и отдельных файлов. Файлы могут иметь все основные присущие им атрибуты (размер, расширение, дата создания и т.д.).

```
================================================
ФАЙЛОВАЯ СИСТЕМА
================================================
1. Создать файл
2. Создать директорию
3. Добавить элемент в директорию
4. Удалить элемент
5. Показать всю структуру
6. Выход

Выберите действие (1-6): 1
Введите имя файла: фиксики
Введите размер файла (байт): 32
Введите расширение: 180
✅ Файл фиксики.180 создан
Добавить в корень? (да/нет): да
```

```
from abc import ABC, abstractmethod

from datetime import datetime


# ========== АБСТРАКТНЫЙ КОМПОНЕНТ ==========

class FileSystemComponent(ABC):

    def __init__(self, name):

        self.name = name

        self.date_created = datetime.now()


    @abstractmethod

    def get_size(self):

        pass


    @abstractmethod

    def display(self, indent=0):

        pass
```

```
# ========== ЛИСТ - ФАЙЛ ==========
class File(FileSystemComponent):
    def __init__(self, name, size, extension):
        super().__init__(name)
        self.size = size
        self.extension = extension

    def get_size(self):
        return self.size

    def display(self, indent=0):
        print(' ' * indent + f"    {self.name}.{self.ext} ({self.size} байт)")

class Directory(FileSystemComponent):
    def __init__(self, name):
        super().__init__(name)
        self.children = []

    def add(self, component):
        self.children.append(component)

    def remove(self, component):
        self.children.remove(component)

    def get_size(self):
        total = 0
        for child in self.children:
            total += child.get_size()
```

```
        return total

    def display(self, indent=0):
        print(' ' * indent + f"    {self.name}/")
        for child in self.children:
            child.display(indent + 2)

def main():
    root = Directory("root")

    while True:
        print("\n" + "="*50)
        print("ФАЙЛОВАЯ СИСТЕМА")
        print("="*50)
        print("1. Создать файл")
        print("2. Создать директорию")
        print("3. Добавить элемент в директорию")
        print("4. Удалить элемент")
        print("5. Показать всю структуру")
        print("6. Выход")

        choice = input("\nВыберите действие (1-6): ")

        if choice == "1":
            name = input("Введите имя файла: ")
            size = int(input("Введите размер файла (байт): "))
            ext = input("Введите расширение: ")
            file = File(name, size, ext)
            print(f"Файл {name}.{ext} создан")
```

```
    dest = input("Добавить в корень? (да/нет): ")
    if dest.lower() == "да":
        root.add(file)
    else:
        dir_name = input("Введите имя директории: ")
        found = False
        for child in root.children:
            if isinstance(child, Directory) and child.name == dir_name:
                child.add(file)
                found = True
                break
        if not found:
            print(" Директория не найдена, файл добавлен в корень")
            root.add(file)

elif choice == "2":
    name = input("Введите имя директории: ")
    dir_obj = Directory(name)
    root.add(dir_obj)
    print(f" Директория {name} создана")

elif choice == "3":
    dir_name = input("Введите имя директории, куда добавить элемент: ")
    for child in root.children:
        if isinstance(child, Directory) and child.name == dir_name:
            print("1. Добавить существующий файл")
            print("2. Создать новый файл")
            sub_choice = input("Выберите (1-2): ")

            if sub_choice == "1":
```

```
            file_name = input("Введите имя файла: ")
            file_size = int(input("Введите размер: "))
            file_ext = input("Введите расширение: ")
            new_file = File(file_name, file_size, file_ext)
            child.add(new_file)
            print(f"✅ Файл добавлен в {dir_name}")
        elif sub_choice == "2":
            name = input("Введите имя файла: ")
            size = int(input("Введите размер (байт): "))
            ext = input("Введите расширение: ")
            new_file = File(name, size, ext)
            child.add(new_file)
            print(f" Файл {name}.{ext} добавлен в {dir_name}")
        break
    else:
        print(" Директория не найдена")

elif choice == "4":
    name = input("Введите имя элемента для удаления: ")
    found = False
    for child in root.children[:]:
        if child.name == name:
            root.remove(child)
            print(f"{name} удален")
            found = True
            break
    if not found:
        print(" Элемент не найден")

elif choice == "5":
```

```
            print("\n" + "="*50)
            print("СТРУКТУРА ФАЙЛОВОЙ СИСТЕМЫ")
            print("="*50)
            root.display()

        elif choice == "6":
            print("До свидания!")
            break

        else:
            print("Неверный выбор!")

if __name__ == "__main__":
    main()
```

Задание 3

Третья группа заданий (поведенческий паттерн)

Реализовать вывод ФС из 2-й группы заданий. Вывод файлов/директорий должен осуществляться в случайном порядке. Вывести основные атрибуты каждого файла/директории.

```
==================================================
ВЫВОД ФАЙЛОВОЙ СИСТЕМЫ В СЛУЧАЙНОМ ПОРЯДКЕ
(Паттерн Итератор)
==================================================

СОЗДАНА СЛЕДУЮЩАЯ СТРУКТУРА ФС:
root/
├── documents/
│   ├── resume.pdf
│   ├── notes.txt
│   ├── presentation.pptx
│   └── projects/
│       ├── main.py
│       └── utils.py
├── photos/
│   ├── vacation.jpg
│   └── family.png
└── music/
    ├── song1.mp3
    └── song2.mp3

==================================================
ВЫВОД ЭЛЕМЕНТОВ В СЛУЧАЙНОМ ПОРЯДКЕ:
==================================================

──────────────────────────────────────────
 root/
   размер: 25088 байт
   количество элементов: 3
   дата создания: 2026-04-02 17:41:52

──────────────────────────────────────────
 utils.py
   расширение: py
   размер: 1024 байт
   дата создания: 2026-04-02 17:41:52

──────────────────────────────────────────
 resume.pdf
   расширение: pdf
```

```
──────────────────────────────────────────
 music/
   размер: 11264 байт
   количество элементов: 2
   дата создания: 2026-04-02 17:41:52

──────────────────────────────────────────
 projects/
   размер: 3072 байт
   количество элементов: 2
   дата создания: 2026-04-02 17:41:52

──────────────────────────────────────────
 documents/
   размер: 6656 байт
   количество элементов: 4
   дата создания: 2026-04-02 17:41:52

──────────────────────────────────────────

Всего выведено элементов: 14
```

```
from abc import ABC, abstractmethod
from datetime import datetime
import random
class Iterator(ABC):
    @abstractmethod
    def has_next(self):
        pass

    @abstractmethod
    def next(self):
        pass

class FileSystemComponent(ABC):
    def __init__(self, name):
```

```
        self.name = name
        self.date_created = datetime.now()

    @abstractmethod
    def get_size(self):
        pass

    @abstractmethod
    def get_attributes(self):
        pass


class File(FileSystemComponent):
    def __init__(self, name, size, extension):
        super().__init__(name)
        self.size = size
        self.extension = extension

    def get_size(self):
        return self.size

    def get_attributes(self):
        return {
            "тип": "файл",
            "имя": self.name,
            "расширение": self.extension,
            "размер": f"{self.size} байт",
            "дата создания": self.date_created.strftime("%Y-%m-%d %H:%M:%S")
        }
```

```
class Directory(FileSystemComponent):
    def __init__(self, name):
        super().__init__(name)
        self.children = []

    def add(self, component):
        self.children.append(component)

    def get_size(self):
        total = 0
        for child in self.children:
            total += child.get_size()
        return total

    def get_attributes(self):
        return {
            "тип": "директория",
            "имя": self.name,
            "размер": f"{self.get_size()} байт",
            "количество элементов": len(self.children),
            "дата создания": self.date_created.strftime("%Y-%m-%d %H:%M:%S")
        }


# ИТЕРАТОР ДЛЯ СЛУЧАЙНОГО ОБХОДА
class RandomIterator(Iterator):
    def __init__(self, root):
        self.items = []
        self._collect_all(root)
```

```
        random.shuffle(self.items)
        self.index = 0

    def _collect_all(self, component):
        """Собирает все элементы ФС"""
        self.items.append(component)
        if isinstance(component, Directory):
            for child in component.children:
                self._collect_all(child)

    def has_next(self):
        return self.index < len(self.items)

    def next(self):
        if self.has_next():
            item = self.items[self.index]
            self.index += 1
            return item
        return None


def main():
    print("="*60)
    print("ВЫВОД ФАЙЛОВОЙ СИСТЕМЫ В СЛУЧАЙНОМ ПОРЯДКЕ")
    print("(Паттерн Итератор)")
    print("="*60)

    root = Directory("root")

    documents = Directory("documents")
```

```
documents.add(File("resume", 1024, "pdf"))
documents.add(File("notes", 512, "txt"))
documents.add(File("presentation", 2048, "pptx"))
root.add(documents)

photos = Directory("photos")
photos.add(File("vacation", 3072, "jpg"))
photos.add(File("family", 4096, "png"))
root.add(photos)

music = Directory("music")
music.add(File("song1", 5120, "mp3"))
music.add(File("song2", 6144, "mp3"))
root.add(music)

projects = Directory("projects")
projects.add(File("main", 2048, "py"))
projects.add(File("utils", 1024, "py"))
documents.add(projects)

print("\nСОЗДАНА СЛЕДУЮЩАЯ СТРУКТУРА ФС:")
print("root/")
print("  ├── documents/")
print("  │   ├── resume.pdf")
print("  │   ├── notes.txt")
print("  │   ├── presentation.pptx")
print("  │   └── projects/")
print("  │       ├── main.py")
print("  │       └── utils.py")
print("  ├── photos/")
```

```
print("  │   ├── vacation.jpg")
print("  │   └── family.png")
print("  └── music/")
print("      ├── song1.mp3")
print("      └── song2.mp3")

print("\n" + "="*60)
print("ВЫВОД ЭЛЕМЕНТОВ В СЛУЧАЙНОМ ПОРЯДКЕ:")
print("="*60)

iterator = RandomIterator(root)

while iterator.has_next():
    element = iterator.next()
    attrs = element.get_attributes()

    print(f"\n{'─'*40}")
    if attrs["тип"] == "файл":
        print(f"  {attrs['имя']}.{attrs['расширение']}")
    else:
        print(f"  {attrs['имя']}/")

    for key, value in attrs.items():
        if key != "тип" and key != "имя":
            print(f"    {key}: {value}")

print(f"\n{'─'*40}")
print(f"\n Всего выведено элементов: {len(iterator.items)}")
```

```
if __name__ == "__main__":

    main()
```